Bharat Ratna Indira Gandhi College of Engineering, Solapur

Master of Computer Application

SUBJECT:

FULL STACK DEVELOPMENT (MCAC301)

SEMESTER-III

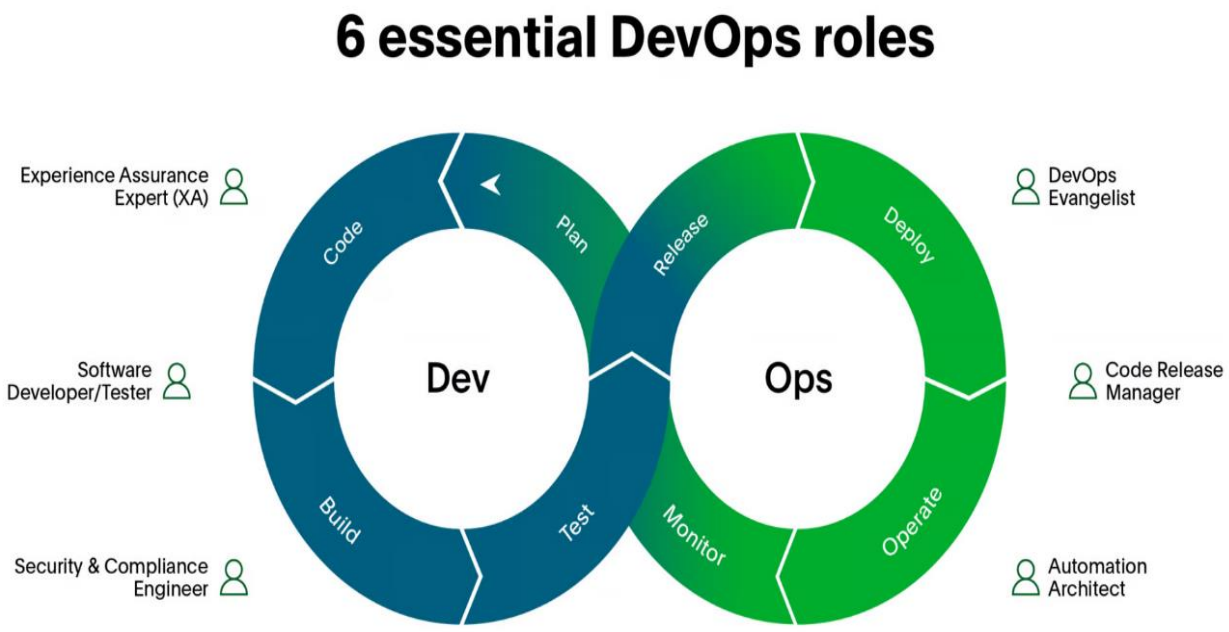
UNIT 4: DEVOPS PRINCIPLES AND PRACTICES

1. Introduction to DevOps and CI/CD

1.1 Overview of DevOps

Definition and Goals

- **Definition**: DevOps is a cultural and professional movement that aims to improve collaboration between development (Dev) and operations (Ops) teams. It seeks to automate and integrate the processes of software development and IT operations to improve the quality, speed, and efficiency of delivering software.
- **Goals**:
    - **Faster Delivery**: Increase the speed of software delivery to meet market demands and customer needs more effectively.
    - **Enhanced Collaboration**: Foster closer collaboration between development and operations teams to reduce friction and improve workflows.
    - **Improved Quality**: Implement practices that ensure higher quality software through continuous feedback and iterative improvements.
    - **Increased Reliability**: Achieve more stable and reliable software releases through automated testing and deployment processes.

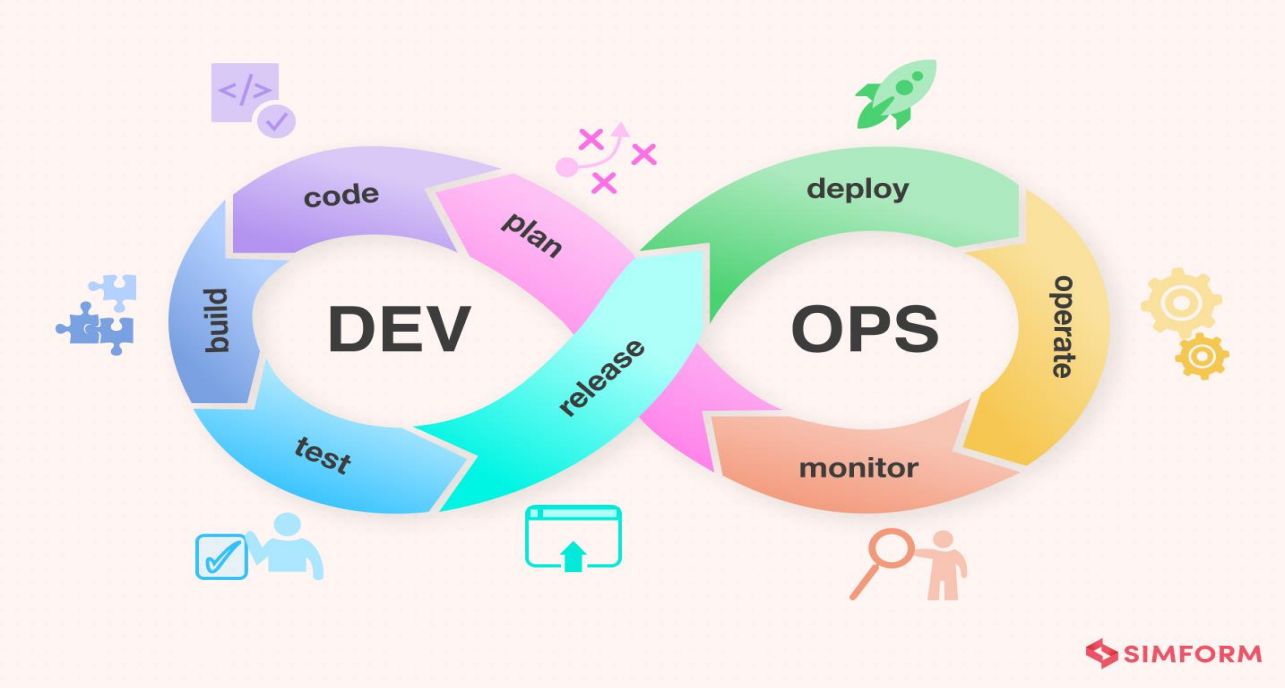


History and Evolution

- **Traditional Software Development**: Characterized by long release cycles, siloed teams, and infrequent integration of code changes, often leading to "integration hell."
- **Agile Development**: Introduced iterative development, continuous feedback, and more frequent releases. Emphasized collaboration between developers and stakeholders but often lacked integration with operations.
- **DevOps Movement**: Emerged as a response to the limitations of Agile by integrating development and operations practices, emphasizing automation, continuous integration, and continuous delivery.

Benefits and Challenges

- **Benefits**:
    - **Faster Delivery**: Shortened release cycles and quicker time-to-market.
    - **Improved Collaboration**: Enhanced communication and cooperation between teams.
    - **Greater Efficiency**: Automation of repetitive tasks and processes.
    - **Higher Quality**: Better testing and more reliable releases.
- **Challenges**:
    - **Cultural Shifts**: Overcoming traditional mindsets and fostering a culture of collaboration.
    - **Tool Integration**: Integrating various tools and technologies used by development and operations teams.
    - **Skill Gaps**: Bridging the gap between different skill sets and knowledge areas.



1.2 Core DevOps Principles

Collaboration

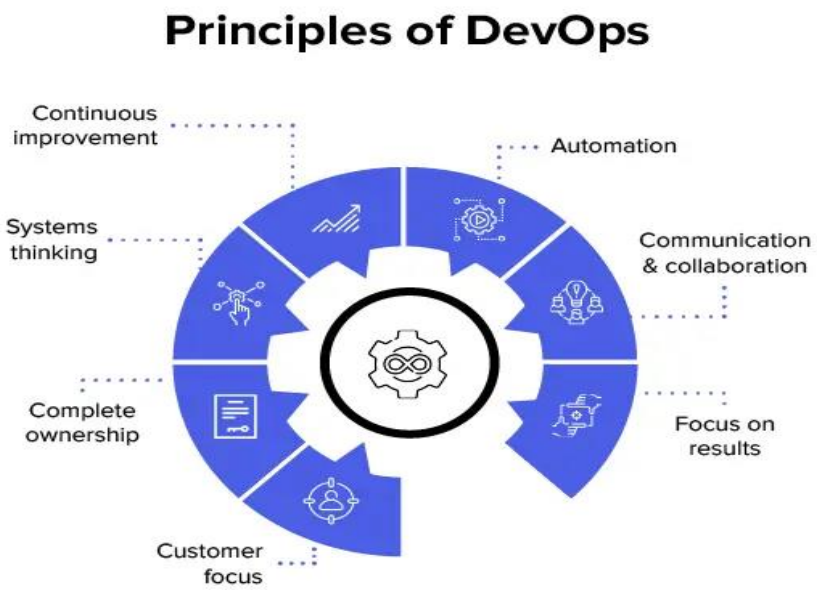
- **Breaking Down Silos**: Promote cross-functional teams where developers, operations, and other stakeholders work together towards common goals.
- **Shared Responsibilities**: Encourage shared ownership of the software lifecycle, from development to deployment and operations.
- **Communication Tools**: Utilize tools such as Slack, Microsoft Teams, and Jira to facilitate communication and collaboration.

Automation

- **Continuous Integration**: Automate the process of integrating code changes into a shared repository to detect issues early.
- **Automated Testing**: Implement automated tests to ensure code quality and catch bugs before deployment.
- **Automated Deployment**: Use deployment pipelines to automate the process of releasing software to production environments.

Continuous Improvement

- **Feedback Loops**: Establish mechanisms for continuous feedback from stakeholders, users, and monitoring tools to drive iterative improvements.
- **Metrics and Monitoring**: Use metrics and monitoring tools to track performance, identify bottlenecks, and make data-driven decisions.
- **Iterative Development**: Embrace iterative cycles for development and deployment, allowing for incremental improvements and rapid adjustments.



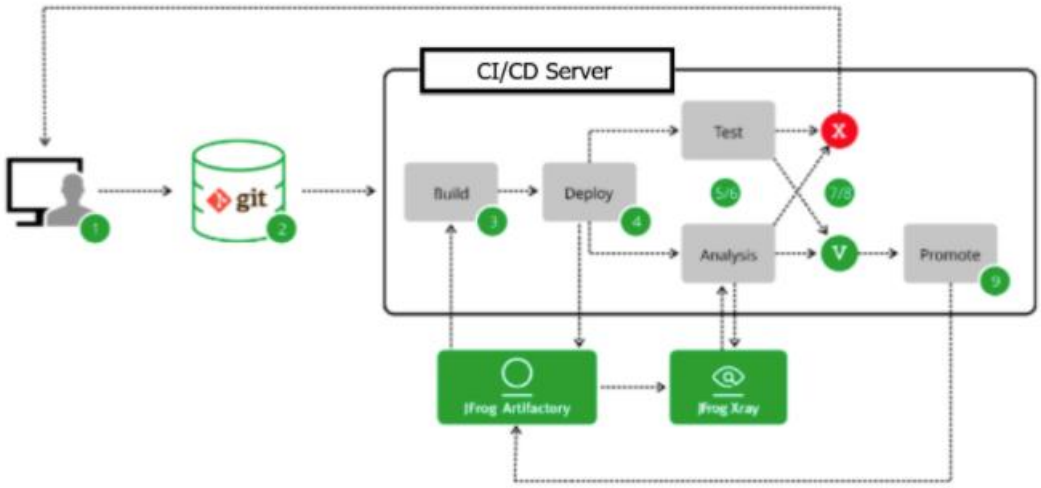
1.3 Introduction to CI/CD

Continuous Integration (CI)

- **Definition**: CI is the practice of frequently integrating code changes into a shared repository. Each integration is verified by an automated build and tests to detect issues early.
- **Benefits**:
    - **Early Detection of Issues**: Immediate feedback on code changes helps identify and fix integration issues quickly.
    - **Reduced Integration Problems**: Frequent integrations minimize the complexity of integrating large changes.
    - **Faster Release Cycles**: Shorter development cycles lead to quicker releases and updates.
- **Tools**:
    - **Jenkins**: An open-source automation server used for building, deploying, and automating projects.
    - **GitLab CI**: A built-in CI/CD tool within GitLab that allows for seamless integration with Git repositories.
    - **Travis CI**: A CI service used to build and test code changes in GitHub repositories.
    - **CircleCI**: A cloud-based CI tool that offers automated testing and deployment for software projects.

Automated Testing

- **Types of Tests**:
    - **Unit Tests**: Test individual components or functions in isolation.
    - **Integration Tests**: Test the interaction between integrated components or systems.
    - **End-to-End Tests**: Test the entire application flow from start to finish.
- **Test Automation Tools**:
    - **Selenium**: An open-source tool for automating web browsers, useful for end-to-end testing.
    - **JUnit**: A framework for writing and running unit tests in Java.
    - **NUnit**: A unit-testing framework for .NET languages.
    - **PyTest**: A testing framework for Python that supports fixtures, parameterized testing, and more.
- **Best Practices**:
    - **Write Effective Tests**: Ensure tests are meaningful and cover various scenarios.
    - **Maintain Test Suites**: Regularly update and refactor test suites to keep them relevant and efficient.
    - **Ensure Test Reliability**: Avoid flaky tests and ensure consistent test results.



Continuous Deployment (CD)

- **Definition**: CD is the practice of automatically deploying code changes to production after passing automated tests, without manual intervention.
- **Benefits**:
  - **Reduced Manual Effort**: Minimizes the need for manual deployment processes.
  - **Faster Release Cycles**: Accelerates the deployment of new features and bug fixes.
- **Tools**:
  - **Jenkins**: Can be configured to handle continuous deployment pipelines.
  - **GitLab CI/CD**: Provides built-in CD capabilities for deploying applications.
  - **Spinnaker**: An open-source platform for continuous delivery and multi-cloud deployments.

2. Continuous Delivery (CD) and Deployment Infrastructure as Code (IaC)

2.1 Continuous Delivery (CD)

Definition and Objectives

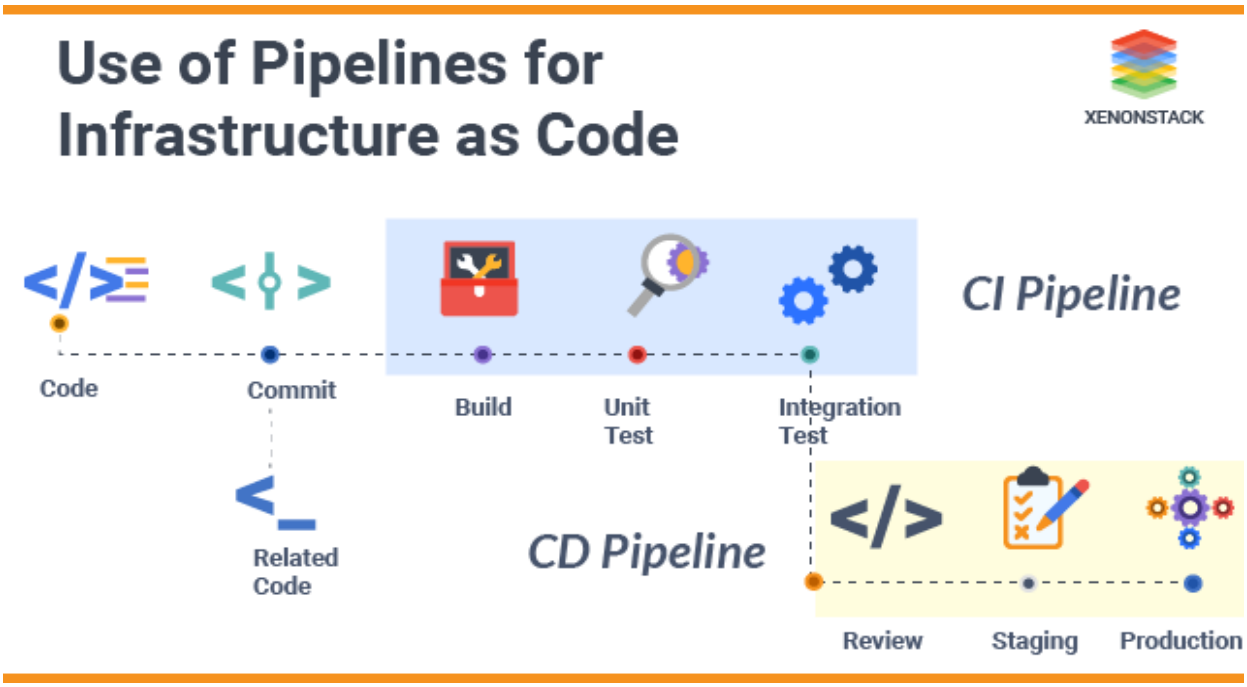
- **Definition**: CD ensures that software can be reliably released at any time by automating the deployment process and maintaining a production-ready state.
- **Objectives**:
  - **Reliable Releases**: Automate deployment processes to ensure consistent and reliable releases.
  - **Quick Rollbacks**: Enable rapid rollback to previous versions in case of issues.
  - **Reduced Risk**: Minimize deployment risks by maintaining a consistent production-ready state.

Deployment Strategies

- **Blue-Green Deployments**:
  - **Description**: Maintain two identical environments (Blue and Green). Deploy the new version to the inactive environment and switch traffic to it once it's validated.
  - **Benefits**: Zero downtime, easy rollback.

- **Canary Releases**:
  - **Description**: Gradually roll out changes to a small subset of users before full deployment.
  - **Benefits**: Allows monitoring of new changes and limits impact if issues arise.

- **Rolling Deployments**:
  - **Description**: Incrementally update instances of an application, replacing old versions with new ones in phases.
  - **Benefits**: Minimizes downtime and allows for continuous availability.



2.2 Infrastructure as Code (IaC)

Concept and Benefits

- **Concept**: IaC involves managing and provisioning computing infrastructure through code rather than manual processes. This approach enables consistent and repeatable deployments.

- **Benefits**:
  - **Repeatability**: Ensure consistent infrastructure setup across environments.
  - **Scalability**: Easily scale infrastructure up or down as needed.
  - **Version Control**: Track changes to infrastructure configurations using version control systems.

	CloudFormation	Terraform
PROS	UI (debugging, overview) Cross Referencing CFN Init Syntax	Open Source Modules Refresh (reconciliation) Count
CONS	Verbose Repeated Code Support (not open source) Confused CLI	Debugging Splitting stack is hard Syntax

Tools and Technologies

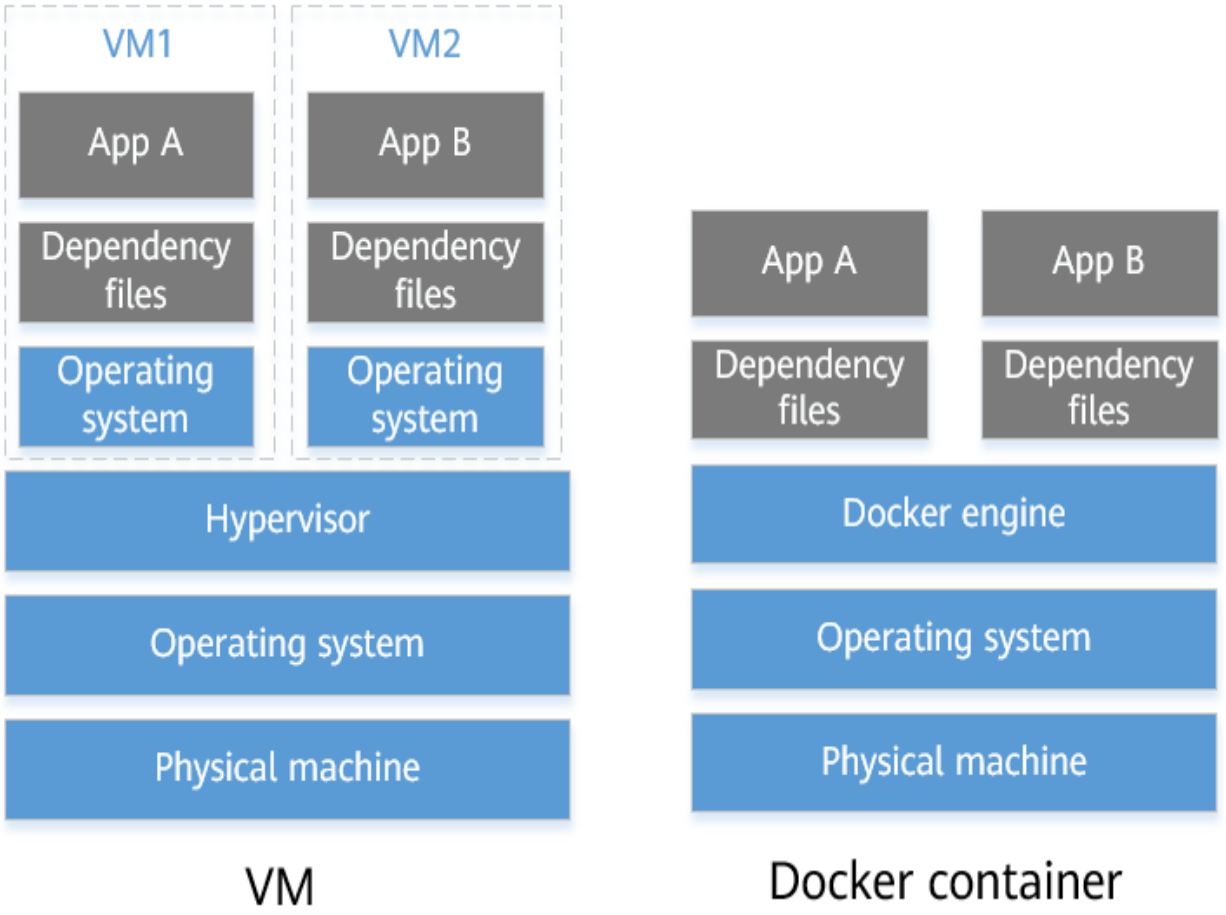
- **Terraform**:
  - **Overview**: An open-source tool for defining and provisioning cloud infrastructure using a declarative configuration language.
  - **Basic Commands**:
    - `terraform init`: Initialize a new or existing Terraform configuration.
    - `terraform plan`: Create an execution plan showing what actions will be taken.
    - `terraform apply`: Apply the changes required to reach the desired state.
  - **Example Configuration**: Define resources like EC2 instances, S3 buckets, and RDS databases using HCL (HashiCorp Configuration Language).

- **AWS CloudFormation**:
  - **Overview**: AWS's native IaC service for defining and provisioning AWS resources using JSON or YAML templates.
  - **Basic Commands**:
    - `aws cloudformation create-stack`: Create a new stack using a template.
    - `aws cloudformation update-stack`: Update an existing stack with a new template.
  - **Example Template**: JSON or YAML template to define resources such as EC2 instances, RDS databases, and security groups.

Comparison Table of Terraform vs. CloudFormation

Criteria	Terraform	CloudFormation
Scope	It covers most AWS resources and parts and is comparatively faster in supporting new AWS features.	It covers major AWS parts but is slow in supporting new AWS capabilities.
License	It is an open-source project.	It is offered by AWS for free.
Support	Hashicorp offers 24*7 support.	AWS support plans that offer CloudFormation support.
State Management	It stores the state over disk and allows using remote state.	It stores and manages the state with the use of stacks.
Modularity	It offers complete native support for modules.	It does not support the use of modules but has features to modularize templates.
Change Verification	It uses a command 'plan' for identifying needed changes.	It uses 'change sets' to verify the required or implemented changes.

3. Containerization and Docker



3.1 Containerization Basics

Definition

- **Definition**: Containerization involves packaging an application and its dependencies into a single, portable container image. Containers ensure that applications run consistently across different environments by isolating them from the underlying infrastructure.

Benefits

- **Portability**: Run containers across various environments (development, testing, production) without modification.
- **Consistency**: Ensure consistent behavior across different deployment environments.
- **Isolation**: Isolate applications and dependencies, reducing conflicts and dependency issues.

CLOUDWITHEASE
A Rendezvous with Cloud Technologies

COMPARISON TABLE: CDK vs Terraform vs CloudFormation

Parameter	CDK	Terraform	CloudFormation
Language And Ease Of Use	JAVA or JASON	• Define resources – JASON like language • Other data - HCL language	YAML or JASON
Scope	Limited to support AWS only	Multi-cloud providers like AZURE, GCP including AWS	AWS, Azure, Google Cloud, and any other provider with a provisioning API
Performance	Most time consuming (CDK code is first converted to CloudFormation templates before application)	Faster	Fast (need some time to support new service capabilities)
Modularity	Can create reusable functions classes and share this code within organization	Native support for modules and also has public module registry for hosting and to use public modules	Does not allow to create and reuse code as modules and it allows to use nested stacks as module
Cloud Resource Management	It synthesizes and leverage all state management and benefits of CloudFormation	Allows to control which resources users can create using a policy as a code tool Sentinel to apply fine grained, logic-based policies to allow/deny user actions	It does not offer any control on how templates are used but you can use AWS IAM policies to allow users to use only standard templates
State Management	The state is stored and maintained by CloudFormation	Stores the state of infrastructure in a state file which is stored locally by default but it can be stored remotely on backend like S3	Does not maintain state file at least not the one you see
Which One To Choose?	If only want to utilize AWS for infrastructure	When using multiple cloud providers/many different resources	If only want to utilize AWS for infrastructure or for simple serverless architecture

3.2 Docker Fundamentals

Architecture

- **Docker Engine**: The core component that runs and manages Docker containers.
- **Docker Hub**: A cloud-based repository for sharing and managing Docker images.
- **Docker Images**: Read-only templates used to create Docker containers. Images include the application and its dependencies.
- **Docker Containers**: Instances of Docker images that run applications in an isolated environment.

Core Commands

- **`docker run`**: Execute a command in a new container from an image.

  Example: `docker run -d -p 80:80 nginx` starts an Nginx container.

- **`docker build`**: Create a Docker image from a Dockerfile.

  Example: `docker build -t myapp`

  builds an image named `myapp` from the current directory.

- **`docker-compose`**: Define and manage multi-container Docker applications using a `docker-compose.yml` file.

  Example: `docker-compose up` starts all containers defined in the file.

Dockerfile

- **Definition**: A Dockerfile is a script with instructions for building a Docker image. It specifies the base image, application dependencies, and configuration.
- **Example**:

dockerfile
Copy code
`# Use an official Node.js runtime as a parent image`
`FROM node:14`
`# Set the working directory`
`WORKDIR /usr/src/app`
`# Copy package.json and install dependencies`
`COPY package*.json ./`
`RUN npm install`
`# Copy the rest of the application code`

```
COPY . .

# Expose the port the app runs on
EXPOSE 8080

# Command to run the application
CMD [ "node", "app.js" ]
```

---

4. Orchestration with Kubernetes

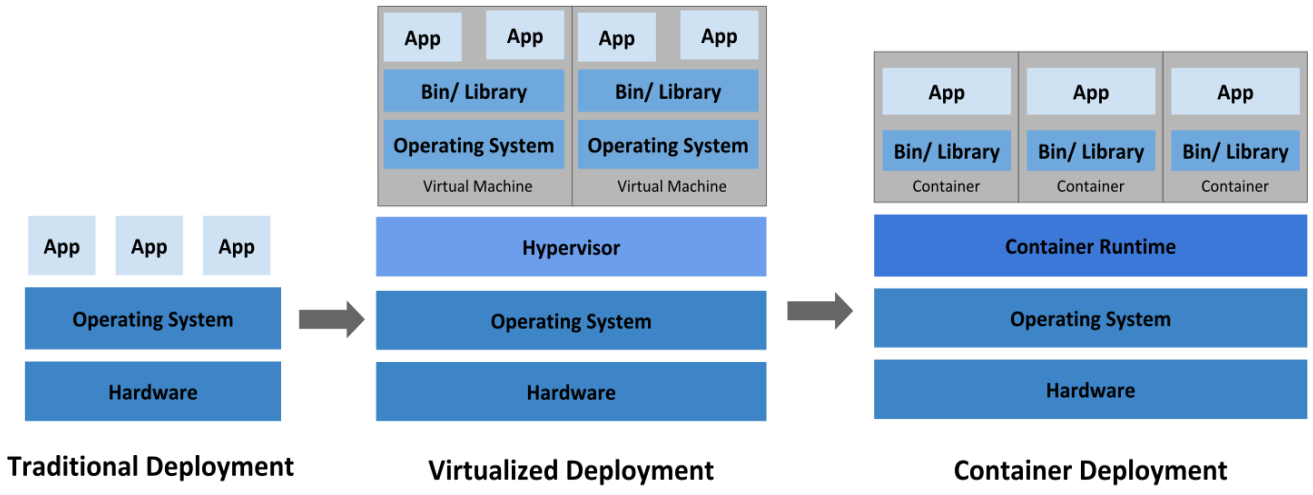
4.1 Kubernetes Overview

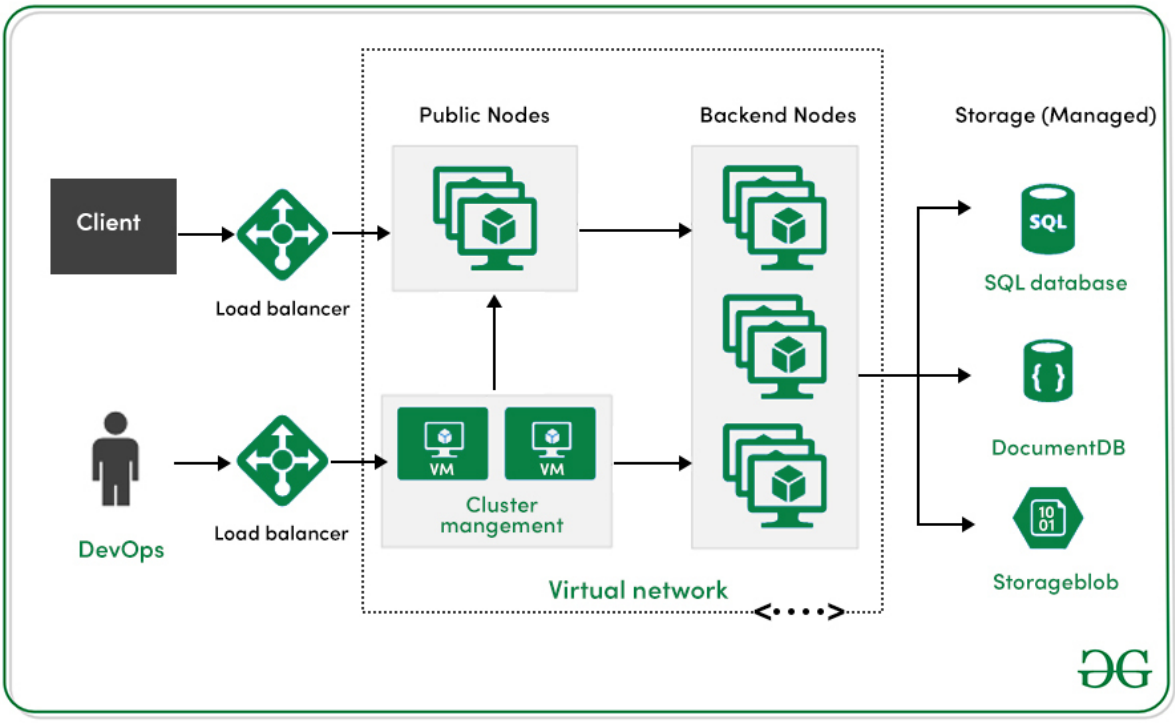
Definition

- **Definition**: Kubernetes is an open-source platform for automating the deployment, scaling, and management of containerized applications. It provides a unified API for managing containerized workloads and services.

Architecture

- **Master Node**: Manages the Kubernetes cluster and controls the worker nodes.
- **Worker Nodes**: Run the containerized applications and include the kubelet, which communicates with the master node.
- **Components**:
  - **Pods**: The smallest deployable units in Kubernetes, which can contain one or more containers.
  - **Services**: Provide a stable network endpoint for accessing a set of Pods.
  - **Deployments**: Manage the deployment and scaling of Pods.
  - **ReplicaSets**: Ensure that a specified number of Pods are running at any given time.
  - **Namespaces**: Provide a way to divide cluster resources between multiple users or projects.





4.2 Core Concepts and Commands

Kubernetes Objects

- **Pods**: The basic execution unit in Kubernetes. A Pod wraps one or more containers and provides networking and storage for them.
- **Services**: A Kubernetes Service abstracts a set of Pods and provides a stable IP address and DNS name for accessing them.
- **Deployments**: Manage the deployment and scaling of applications by controlling the desired state and rolling updates.
- **ConfigMaps and Secrets**: Manage configuration data and sensitive information separately from application code.

Basic Commands

- **`kubectl get`**: List resources. Example: `kubectl get pods` lists all Pods in the current namespace.
- **`kubectl apply`**: Apply a configuration to a resource.

  Example: `kubectl apply -f deployment.yaml`

  (applies a deployment configuration.)

- **`kubectl describe`**: Show detailed information about a resource.

  Example: `kubectl describe pod my-pod`
  (provides detailed information about a specific Pod.)

4.3 Helm (Optional)

Overview

- **Overview**: Helm is a package manager for Kubernetes that simplifies the deployment and management of complex applications by using pre-defined charts (packaged configurations).

Basic Commands

- **`helm install`**: Install a Helm chart.

  Example: `helm install my-release stable/mysql` installs the MySQL chart with the release name `my-release`.

- **`helm upgrade`**: Upgrade a release to a new version.

  Example: `helm upgrade my-release stable/mysql` upgrades the release to a new version of the chart.

- **`helm rollback`**: Rollback a release to a previous version.

  Example: `helm rollback my-release 1` rolls back the release to revision 1.